

Setup 1 — The Zephyr SDK Toolchain

Goal

Understand what the Zephyr SDK is, what is inside it, and how each component is used during a build and flash cycle.

What you will learn

- The difference between the SDK (toolchain) and the Zephyr source code
- What `arm-zephyr-eabi-gcc` is and why it is different from your host GCC
- Which SDK tools are used at build time versus flash time
- Why SDK version 1.0.1 is required for Zephyr 4.x

Overview

The Zephyr SDK is installed once in your home directory (`~/zephyr-sdk-1.0.1/`) and is completely separate from the Zephyr OS source code. It contains only **build tools** — it has no RTOS kernel or driver code inside it.

When you run `west build`, CMake finds the SDK automatically and uses its compiler to turn your C source and the Zephyr kernel into ARM machine code.

What is inside the SDK

```
~/zephyr-sdk-1.0.1/
├─ gcc/                               ← cross-compilers for every
supported architecture
|   └─ arm-zephyr-eabi/               ← the one used for nRF52840
(Cortex-M4)
|   |   └─ bin/
```

```

| | | └─ arm-zephyr-eabi-gcc          ← C compiler
| | | └─ arm-zephyr-eabi-g++         ← C++ compiler
| | | └─ arm-zephyr-eabi-as          ← assembler
| | | └─ arm-zephyr-eabi-ld          ← linker
| | | └─ arm-zephyr-eabi-ar          ← static library
archiver
| | | └─ arm-zephyr-eabi-objcopy     ← converts ELF
→ .bin / .hex
| | | └─ arm-zephyr-eabi-nm          ← lists symbols in
object files
| | | └─ arm-zephyr-eabi-gdb          ← interactive debugger
| | └─ aarch64-zephyr-elf/           ← for 64-bit ARM (not used here)
| | └─ riscv64-zephyr-elf/           ← for RISC-V boards
| | └─ ...                           ← 30+ other architectures
└─ hosttools/                        ← host-side helper tools
    └─ zephyr-sdk-x86_64-hosttools-standalone-0.10.sh
└─ cmake/                            ← CMake modules that teach west
where the SDK is
└─ sdk_version                       ← version file CMake reads to
check compatibility
└─ setup.sh                          ← registers the SDK with CMake on
first install

```

The ARM cross-compiler: arm-zephyr-eabi-gcc

Your host PC runs x86-64 Linux. The nRF52840 is an ARM Cortex-M4. Code compiled for x86-64 will not run on ARM — the instruction sets are different. A **cross-compiler** runs on x86-64 but produces ARM machine code.

arm-zephyr-eabi-gcc is GCC 14.3.0 configured as an ARM cross-compiler. The name breaks down as:

Part	Meaning
arm	target architecture: ARM
zephyr	vendor: Zephyr project (custom sysroot/ libc)
eabi	ABI: Embedded Application Binary Interface (no OS, no dynamic linking)
gcc	tool: GNU C Compiler

Every other tool in `arm-zephyr-eabi/bin/` follows the same naming prefix and targets the same ARM EABI.

Build-time tools (used by `west build`)

Tool	Role
<code>arm-zephyr-eabi-gcc</code>	Compiles <code>.c</code> files → <code>.o</code> object files
<code>arm-zephyr-eabi-as</code>	Assembles <code>.S</code> startup files → <code>.o</code>
<code>arm-zephyr-eabi-ld</code>	Links all <code>.o</code> files → <code>zephyr.elf</code>
<code>arm-zephyr-eabi-objcopy</code>	Converts <code>zephyr.elf</code> → <code>zephyr.bin</code> and <code>zephyr.hex</code> for flashing
<code>arm-zephyr-eabi-nm</code>	Lists symbol sizes (used for map file analysis)

Flash-time tools (used by `west flash`)

`west flash` calls **J-Link** (Nordic's on-board debugger chip on the DK) to write `zephyr.hex` to the nRF52840's internal flash. The SDK sets up the path to JLinkExe via the CMake modules in `cmake/`.

Why version 1.0.1?

Zephyr's CMake build system checks the SDK version at configure time:

```
CMake Error: Could NOT find Zephyr-sdk compatible with version 1.0
```

SDK 0.16.x only supports Zephyr 3.x. SDK 1.0.1 is required for Zephyr 4.x. The check reads `~/zephyr-sdk-1.0.1/sdk_version` and compares it against the minimum version declared inside `myworkspace/zephyr/cmake/modules/FindZephyr-sdk.cmake`.

How CMake finds the SDK

After running `./setup.sh`, the SDK registers itself in `~/cmake/packages/Zephyr-sdk/`. CMake's `find_package(Zephyr-sdk)` searches that registry at build time — you do not need to set any environment variable pointing to the SDK.

Why this matters

Confusing the SDK (toolchain) with the Zephyr source is a common source of errors. The SDK is installed once; the Zephyr source is fetched per-workspace with `west update`. Upgrading one does not upgrade the other.

Practice tasks

1. Run `arm-zephyr-eabi-gcc --version` after adding the SDK to your PATH — confirm it reports GCC 14.3.0 for arm-zephyr-eabi.
2. After a successful build, run `arm-zephyr-eabi-nm build/zephyr/zephyr.elf | sort -k1 | tail -20` to see the largest symbols in your firmware.
3. Run `arm-zephyr-eabi-objcopy --info` to see the supported output formats (BIN, HEX, ELF, SREC, etc.).